Classifying superimposed image text

Ofir Yeshurun

Summary. A pipeline classifying images as `alcohol` or `non_alcohol` based on superimposed text. Images are contrast-enhanced with LAB-CLAHE and parsed by PaddleOCR (PP-OCRv6 small); if OCR confidence drops below 0.9, OCR re-runs on a horizontally mirrored image. Extracted text is then cleaned and scored with a character *n*-gram TF-IDF logistic regression classifier.

1 Data exploration

For the analysis, I annotated 60 training images (30 per class) to record their superimposed text.

- The dataset is balanced, with 550 images per class. It is not redistributed with the code.
- Each superimposed text holds one key term (alcoholic or not), often inside a short sentence ("Try event imported *lager* premium", Figure 1a).
- Some alcohol terms are drink names (Figure 1a); others are brand names only (Figure 1b).
- Non-alcohol cases are near-domain: other beverages and cannabis-related terms (Figure 1c, d).
- Both classes contain text distortions: mirroring, rotation, split characters, etc. (Figure 1b, e-h).
- Text is placed either inside a solid bounding box (Figure 1a) or directly over the image (Figure 1b).
- Some photos contain their own text, such as signs and labels (Figure 1i).



Figure 1: Train examples: (a, b, e-h) alcohol, (c, d, i) non-alcohol. (i) The superimposed text reads "Looking for brew sparkling water check", but the photo's sign says "cold beer".

2 Methodology

Assumptions. Framing the classifier as a content-moderation filter, my main assumption was that a missed alcohol image is more harmful than a false positive, so throughout the project recall on alcohol was the primary metric.

Protocol. A stratified 70/15/15 split (seeded). Every choice was made by stratified 5-fold cross-validation. The test split was scored once, by the inference script (`scripts/inference.py`).

Pipeline (Figure 2).

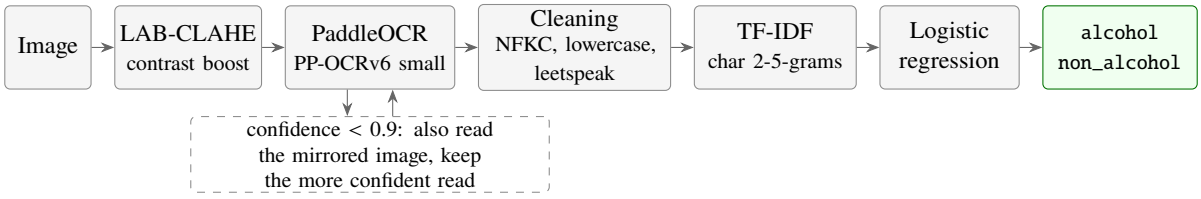


Figure 2: The shipped pipeline.

3 Experiments

Choosing the OCR engine. On the annotated sample, scored by *key-term hit rate* (the share of images whose key term the OCR read), `PP-OCRv6-medium` led on raw images with 0.67, over `v6-small` (0.58) and `PP-OCRv5-mobile` (0.50).
Most missed cases were mirrored text, so I added a retry on the mirrored image when OCR confidence dropped below 0.9. This increased the hit rate to 0.87 (`v6-medium`) and 0.82 (`v6-small`), stable across several confidence thresholds.

Rotated and faint text. The same retry mechanism applied to rotations (±20°) added no alcohol hits and more than doubled the extraction time per image. I tried several preprocessing methods to improve reads of faint text (CLAHE variants, top-hat and others); each improved a few images but worsened more of them. The exception was LAB-CLAHE: on v6-small it fixed a few alcohol images and broke none.

Classifiers.

- *v0, lexicon matching:* fuzzy matching against a domain-knowledge lexicon of alcohol terms, with an error margin that grows with term length.
- *v1, character 2-5-gram TF-IDF + logistic regression.* Error analysis showed that OCR mostly dropped characters, and short letter chunks survive that.
- *v2, frozen MiniLM embeddings + logistic regression.* Embeddings should carry knowledge to brands unseen in training.

v1 led on every OCR setup. v2 trailed, I assume because near-domain negatives (lemonade, espresso or CBD) sit close to alcohol in the embedding space. For v1 the OCR setups overlap. However, small + LAB-CLAHE had the highest recall at a quarter of medium's image processing time (Table 1).

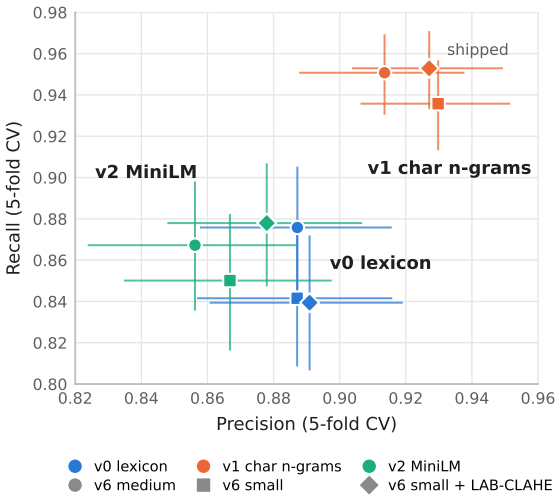


Figure 3: Recall vs precision with 95% intervals, per classifier and OCR setup.

Table 1: OCR engine trade-off, best setup per PP-OCRv6 engine.

OCR setup	Key-term hit*	Alcohol terms read*	Text found	Recall	Precision	s/img
Medium, raw image	0.867	28/30	98.4%	0.951	0.914	3.31
Small + LAB CLAHE (shipped)	0.850	25/30	97.1%	**0.953**	**0.927**	**0.83**

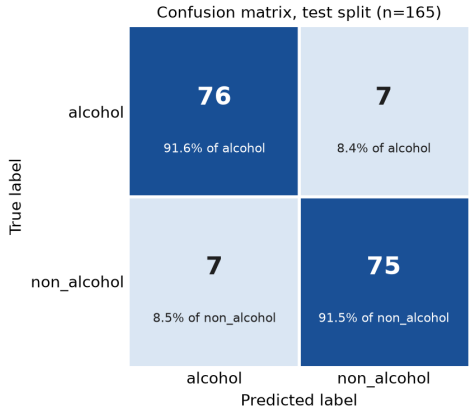
*On the annotated sample. Other columns: all train+val images; recall and precision are v1's CV scores.

4 Final performance

Table 2: Test split.

	Test, 165 images	CV, 935 images
Recall	**0.916** [0.852, 0.973]	0.953 [0.933, 0.971]
Precision	**0.916** [0.853, 0.974]	0.927 [0.904, 0.949]
F1	0.916 [0.867, 0.956]	0.940 [0.924, 0.955]

On test, recall and precision are both 0.916 (accuracy 0.915): fewer than one in ten alcohol images is missed, and fewer than one in ten non-alcohol images is wrongly flagged. Inference took 0.75 s per image on CPU.



5 Failure cases and ideas for improvement



Figure 4: Examples of test errors: (a)-(c) missed alcohol images, (d) a non-alcohol image flagged as alcohol (false positive).

Table 3: What went wrong in each case, and the fix it points to.

Case	OCR read	Likely reason	Suggested fix
(a) 0.44	`XsNAob16`	OCR missed the faint, semi-transparent text	An extra preprocessing retry on weak reads
(b) 0.48	`Ybnpid`	Rotated text read weakly, so the mirror retry ran and its distorted read won	Keep the mirrored read only when it is clearly more confident
(c) 0.46	`Tryeven tport ToshikiMatsuya Nike ...`	Split letters (“t port”) glued into one word; background text added noise	Re-segment split and glued fragments (`Tryeven tport` → `try event port`, `be er` → `beer`); down-weight scene text
(d) 0.53	`STUPb now ... HARPER END ...`	“milk” lost (red text on a red sign); the sign’s text drives the score	Down-weight scene text; try other preprocessing views